

ASSESSMENT BRIEF 2

COURSE: Master of Information and Communication Technology	
Unit Code:	MICT802
Unit Title:	Development of Advanced Applications
Type of Assessment:	Assessment 2 - Individual Project and Report
Length/Duration:	N.A.
Unit Learning Outcomes addressed:	1. Analyse and apply Python programming techniques to solve real-world, data-driven application problems. 2. Analyse fundamental mathematical and statistical concepts for use in data-driven and advanced software applications. 3. Create data-driven and intelligent applications using Python, including the integration of libraries for data processing and analysis.
Submission Date:	Week 7
Assessment Task:	Assessment 2 - Individual Project and Report (Individual Case Study)
Total Mark:	30 Marks
Weighting:	30% of unit's total marks
Students are advised that submission of an Assessment Task past the due date without a formally signed approved Assignment Extension Form (Kent Website MyKent Student Link> FORM – Assignment Extension Application Form – Student Login Required) or previously approved application for other extenuating circumstances impacting course of study, incurs a 5% penalty per calendar day, calculated by deduction from the <u>total mark</u>. For example. An Assessment Task marked out of 40 will incur a 2 mark penalty <u>for each calendar day</u>. More information, please refer to (Kent Website MyKent Student Link> POLICY – Assessment Policy & Procedures – Student Login Required)	

ASSESSMENT DESCRIPTION:

In this assessment, you are required to design and implement a **Python-based Sales Analysis and Decision Support System** for a small business. The system simulates how a business owner might analyze daily sales figures to understand overall performance and make basic operational decisions.

The program will process a collection of sales values, perform numerical calculations, apply decision-making logic, and present clear, user-friendly results. Students **must use programming concepts covered** including variables, conditionals, loops, functions, and lists.

In addition to implementing the Python program, you are required to submit a **written technical report** explaining the problem, design approach, implementation logic, and results.

Project Scenario

You are developing a simple analysis tool for a **small retail business** that records daily sales amounts. The business owner wants a program that can:

- Process multiple daily sales values
- Ignore invalid sales data
- Calculate key financial figures
- Categorize business performance based on predefined rules

The program must help the business owner quickly understand whether sales performance is **poor, average, or strong**, based on calculated results.

Project Requirements (Programming Component)

You must develop a **fully functional Python program** that meets **all** of the following requirements.

1. Sales Data Input and Storage

- Store daily sales amounts in a **list**
- The list must contain **at least 10 numeric values**
- Sales values represent amounts in dollars and must be **greater than or equal to 0**

You may:

- Use predefined values in the program **OR**
- Allow the user to enter values using input statements

2. Data Validation

Your program must:

- Check that each sales value is **valid (≥ 0)**
- Ignore invalid values and display a warning message when invalid data is encountered
- Only valid sales values should be used in calculations

3. Sales Calculations

Using **basic arithmetic and built-in functions only**, your program must calculate:

- Total sales amount
- Average daily sales (*simple arithmetic average*)
- Highest daily sales
- Lowest daily sales

4. Business Performance Classification

Based on the **average daily sales**, classify business performance using **conditional logic**:

Average Daily Sales (\$)	Performance Level
≥ 1000	Excellent
700 – 999	Good
400 – 699	Average
< 400	Poor

5. Modular Program Design

Your program must use **functions** to organize the solution. At a minimum, functions should be used for:

- Validating sales data
- Calculating total and average sales
- Determining performance category
- Displaying results

6. Iterative Processing

- Use **loops** to process the list of sales values
- Use loops for validation and calculations
- Avoid code repetition by using functions and iteration

7. Output Requirements

Your program must clearly display:

- The list of valid sales values
- Total sales
- Average daily sales
- Highest and lowest sales values
- Final business performance classification

All outputs must be **clearly labelled** and easy for a non-technical user to understand.

Sample Output

Input:

```
sales = [850, 1200, 640, 980, 450, 1100, 720, 300, 900, 1050, -20]
```

Output:

```
Valid Sales Values: [850, 1200, 640, 980, 450, 1100, 720, 300, 900, 1050]  
Total Sales: $8190  
Average Daily Sales: $819.0  
Highest Sale: $1200  
Lowest Sale: $300  
Business Performance Level: Good
```

(Your output values may differ depending on input.)

Report Requirements (1,200–1,500 words)

The written report must include the following sections:

1. Introduction

- Description of the business sales problem
- Importance of analyzing sales data for decision-making
- Overview of the developed solution

2. Problem Analysis

- Description of input data and constraints
- Expected outputs
- Assumptions (e.g., non-negative sales values)
- Limitations of the system

3. Design and Algorithm

- Explanation of the program structure
- Description of functions used
- Explanation of loops and conditional logic

- Rationale for key design decisions

4. Implementation Overview

- [Paste your code here]
- Explanation of key sections of the Python code
- Description of validation and calculation logic
- Explanation of how the program processes data step by step

5. Results and Discussion

- Example program execution
- Interpretation of the output results
- Discussion of edge cases (e.g., invalid inputs)

6. Conclusion

- Summary of what the system achieves
- Reflection on potential future improvements (beyond Week 5 scope)

Submission Guidelines:

- **Submission Format:** One Word document containing code and report.

MARKING RUBRICS

Criterion	Marks	High Distinction (85–100%)	Distinction (75–84%)	Credit (65–74%)	Pass (50–64%)	Fail (0–49%)
Problem Analysis & Understanding	15%	Demonstrates a clear, accurate, and well-justified understanding of the business sales problem, including inputs, outputs, constraints, and assumptions. Strong real-world relevance and analytical reasoning.	Demonstrates good understanding with minor gaps in assumptions or constraints.	Demonstrates basic understanding but analysis lacks depth or clarity in places.	Limited understanding; problem description is vague or partially incorrect.	Little or no evidence of understanding; analysis missing or incorrect.
Program Design & Use of Functions	10%	Program is well structured with multiple clearly defined functions. Excellent modularity, logical decomposition, and clear justification of design decisions.	Good functional decomposition with minor structural or naming issues.	Uses functions but structure is basic or partially repetitive.	Minimal use of functions; most logic placed in main program.	No meaningful use of functions or poor program structure.
Use of Loops, Lists & Conditionals	20%	Correct, efficient, and readable use of lists, loops, and conditionals. Logic handles all required cases accurately.	Mostly correct use with minor logical or efficiency issues.	Basic use of loops and lists; logic works but is simplistic or unclear.	Limited or partially incorrect use of loops and conditionals.	Incorrect or missing use of loops, lists, or conditionals.
Mathematical Logic & Calculations	15%	All calculations (total, average, min, max) are correct, logically implemented, and clearly explained. Handles edge cases appropriately.	Calculations mostly correct with minor errors or limited explanation.	Basic calculations implemented but may contain small errors or unclear logic.	Limited or partially incorrect calculations.	Calculations missing, incorrect, or outside Week 1–5 scope.
Program Correctness & Output Quality	15%	Program runs without errors. Outputs are accurate, clearly labelled, well formatted, and user-friendly. Validation and classification logic work correctly.	Program runs correctly with minor formatting or clarity issues.	Program runs but outputs are partially unclear or slightly incorrect.	Program runs inconsistently or produces confusing outputs.	Program fails to run or produces incorrect or missing outputs.
Report Structure & Technical Communication	15%	Report is well organised, clearly written, and professional. All required sections are present with strong technical explanations.	Report is clear and well structured with minor issues in flow or clarity.	Report includes required sections but explanations lack depth or clarity.	Report is poorly structured or explanations are minimal.	Report missing major sections or is unclear and incoherent.

Code Quality & Documentation	10%	Code is clean, readable, well-commented, and consistently formatted. Variable and function names are meaningful and professional.	Code is readable and commented with minor formatting issues.	Code is understandable but poorly commented or inconsistently formatted.	Code is difficult to read with minimal documentation.	Code is unreadable, undocumented, or poorly written.
------------------------------	-----	---	--	--	---	--

ASSESSMENT SUBMISSION:

The assignment must be submitted online in Moodle. Where additional documents need to be attached, all materials **MUST** be submitted electronically in Microsoft Word format. Other formats (e.g., pdf or MAC file) may not be readable by markers. Please be aware that any assessments submitted in other formats will be considered LATE and will lose marks until it is presented in MS Word. No paper based or hardcopy submission will be accepted.

GENERAL NOTES FOR ASSESSMENT TASKS

Content for Assessment Task papers should incorporate a formal introduction, main points and conclusion.

Appropriate academic writing and referencing are inevitable academic skills that you must develop and demonstrate in work being presented for assessment. The content of high quality work presented by a student must be fully referenced within-text citations and a Reference List at the end. Kent strongly recommends you refer to the Academic Learning Support Workshop materials available on the Kent Learning Management System (Moodle). For details please click the link <http://moodle.kent.edu.au/kentmoodle/mod/folder/view.php?id=3606> and download the file titled "Harvard Referencing Workbook". This Moodle Site is the location for Workbooks and information that are presented to Kent Students in the ALS Workshops conducted at the beginning of each Trimester.

Kent recommends a minimum of **FIVE (5)** references in work being presented for assessment. Unless otherwise specifically instructed by your Lecturer or as detailed in the Unit Outline for the specific Assessment Task, any paper with less than five (5) references may be deemed not meeting a satisfactory standard and possibly be failed.

Content in Assessment tasks that includes sources that are not properly referenced according to the "*Harvard Referencing Workbook*" will be penalised.

Marks will be deducted for failure to adhere to the word count if this is specifically stated for the Assessment Task in the Unit Outline. As a general rule there is an allowable discretionary variance to the word count in that it is generally accepted that a student may go over or under by 10% than the stated length.

AI POLICY:

Permissible Use of AI: Students are permitted to employ AI tools for educational purposes, including research, information gathering, concept clarification, and enhancing their understanding of course material.

Prohibition of Use of AI: Students must not directly copy and paste content generated by AI tools into their assessment tasks, assignments, projects, or examinations. The verbatim use of AI-generated material is strictly prohibited.

Originality and Individual Work: All assessment tasks, including essays, reports, presentations, and projects, must be the original work of the student. The primary content and analysis presented in these tasks should reflect the student's independent understanding and efforts.

GENERAL NOTES FOR REFERENCING

References are assessed for their quality. Students should draw on quality academic sources, such as books, chapters from edited books, journals etc. The textbook for the Unit of study can be used as a reference, but not the Lecturer Notes. The Assessor will want to see evidence that a student is capable of conducting their own research. Also, in order to help Assessors determine a student's understanding of the work they cite, all in-text references (not just direct quotes) must include the specific page number(s) if shown in the original. Before preparing your Assessment Task or own contribution, please review this 'YouTube' video (Avoiding Plagiarism

through Referencing) by clicking on the following link: link:
<http://moodle.kent.edu.au/kentmoodle/mod/folder/view.php?id=3606>

<https://www.youtube.com/watch?v=2q0NIWcTq1Y> A search for peer-reviewed journal articles may also assist students. These type of journal articles can be located in the online journal databases and can be accessed from the Kent Library homepage. Wikipedia, online dictionaries and online encyclopaedias are acceptable as a starting point to gain knowledge about a topic, but should not be over-used – these should constitute no more than 10% of your total list of references/sources. Additional information and literature can be used where these are produced by legitimate sources, such as government departments, research institutes such as the National Health and Medical Research Council (NHMRC), or international organisations such as the World Health Organisation (WHO). Legitimate organisations and government departments produce peer reviewed reports and articles and are therefore very useful and mostly very current. The content of the following link explains why it is not acceptable to use non-peer reviewed websites (Why can't I just Google?): <https://www.youtube.com/watch?v=N39mnu1Pkgw>
_ (thank you to La Trobe University for access to this video).